Galala University
Faculty of Computer Science and Engineering
CSE243 Secure Programming
Spring-2025/2026

Exam Management System

Secure Programming Project Report

Team:

Name	ID	Major
Samaa Mohamed Saleh	223101629	CE - Cybersecurity
Omar Khalifa	223101614	CE - Cybersecurity

Instructor: Dr. Mohamed Mead
CSE243: Secure Programming Course Project

Galala University
Faculty of Computer Science and Engineering
CSE243 Secure Programming
Spring-2025/2026

GALALA UNIVERSITY
Powered by
Arizona State University®

Table Of Contents

1. Introduction **3**
1.1 Project Overview 3
1.2 Technology Stack 3
2. Secure Software Development Life Cycle (SSDLC) **4**
2.1 Secure Requirement Gathering - Abuse Cases & Risk Analysis **4**
2.2 Secure Architecture and Design **5**
5. Threat Diagrams **5**
5.1 Stride threat model 5
5.2 Threat tree diagram attack surface 8
2.3 Threat Modeling - STRIDE on Data Flow Diagrams **10**
2.4 Security Testing **12**
3. Design - Screens & Database **13**
4. Implementation Strategy **13**
6. Demo **15**
6.1 SQL Injection (SQLi) Protection 15
6.2 Cross-Site Scripting (XSS) & Cookie Security 16
6.21 Cookie Accessibility (Console Test) 16
6.22 UI Script Injection 16
6.23 CSRF (Cross-Site Request Forgery) Protection 17
6.3 Brute Force & Rate Limiting 17
6.4 Session Hijacking (Single Session Enforcement) 18
6.5 Data Encryption (At Rest) 18
7. Conclusion **19**

Galala University
Faculty of Computer Science and Engineering
CSE243 Secure Programming
Spring-2025/2026

1. Introduction

1.1 Project Overview

Exam Management System is an online system that digitizes the entire process of conducting exams within an institution, including creation, administration, evaluation, and results reporting. The system defines three main user roles: administrators, teachers, and students. Teachers can create and publish exams consisting of multiple-choice and open-ended questions, configure time limits, randomization settings, and control access windows. Exams are delivered through a secure token-based authentication and session management system that enforces strict time limits and automatically submits and processes answers upon completion or expiration. Automatic grading is applied to objective questions, and performance analytics are displayed through a dashboard interface. Administrators manage users and system configurations, while all functionalities are strictly controlled through role-based access control. The system also includes an end-to-end encrypted chat and access control for secure teacher-student communication.

1.2 Technology Stack

Component	Technology	Security Rationale
Backend	Node.js + Express.js	Non-blocking I/O; large middleware ecosystem; native crypto module eliminates third-party crypto dependencies
Database	SQLite3 (better-sqlite3)	WAL mode for concurrent writes; parameterized queries prevent SQLi; file stored outside web root
Cryptography	Node.js crypto module	No third-party crypto libraries — eliminates supply-chain attack risk; uses OS-level primitives
Templating	EJS (server-side)	Server-side rendering prevents CSP bypass; no client-side template injection risk
Validation	validator.js	Proven sanitization library; escape() + trim() on all user input fields
Sessions	JWT + HttpOnly Cookies	Short-lived tokens (2hr); HttpOnly prevents JS theft; Secure flag enforces HTTPS
Rate Limiting	express-rate-limit	5 attempts/min on auth routes; 100/min general; 429 responses with Retry-After

2. Secure Software Development Life Cycle (SSDLC)

The SSDLC ensures security is integrated at every phase, not bolted on as an afterthought. ExamGuard Pro follows five SSDLC phases with security gates.

2.1 Secure Requirement Gathering — Abuse Cases & Risk Analysis

Abuse Cases

Abuse cases identify how attackers might misuse the system and define countermeasures:

ID	Abuse Case	Mitigation
AC-01	SQL injection on login form to bypass auth	Parameterized queries: db.prepare("SELECT * FROM users WHERE username = ?").get(username)
AC-02	Brute force password guessing	Rate limiting (5 attempts/min) + account lockout after 5 failures + CAPTCHA
AC-03	XSS payload in registration fields	validator.escape() on all input + Content-Security-Policy header
AC-04	Session cookie theft via XSS	HttpOnly + SameSite=Strict + Secure cookie flags
AC-05	CSRF attack on exam submission	Per-request CSRF tokens stored in DB, single-use validation
AC-06	Student navigates away during exam	ABAC middleware auto-submits and flags as security violation
AC-07	Role escalation (student → admin)	RBAC authorize() middleware checks role on every request, logs violations

Risk Analysis Matrix

Risk	Impact	Likelihood	Mitigation
SQL Injection	Critical	High	Parameterized queries for all DB operations
Cross-Site Scripting	High	Medium	Input escaping + CSP headers
Brute Force Attack	High	High	Rate limiting + lockout + CAPTCHA
Session Hijacking	Critical	Medium	HttpOnly/Secure/SameSite cookies
CSRF	High	Medium	Per-request CSRF tokens
Privilege Escalation	Critical	Low	RBAC + ABAC middleware
Data Breach	Critical	Low	AES-256 encryption + PBKDF2 hashing

GALALA UNIVERSITY
Powered by
Arizona State University®

2.2 Secure Architecture and Design

Threat Model Diagram

The threat tree decomposes the top-level threat "Compromise ExamGuard Pro" into sub-threats across authentication, authorization, data, and availability attack vectors.

5. Threat Diagrams

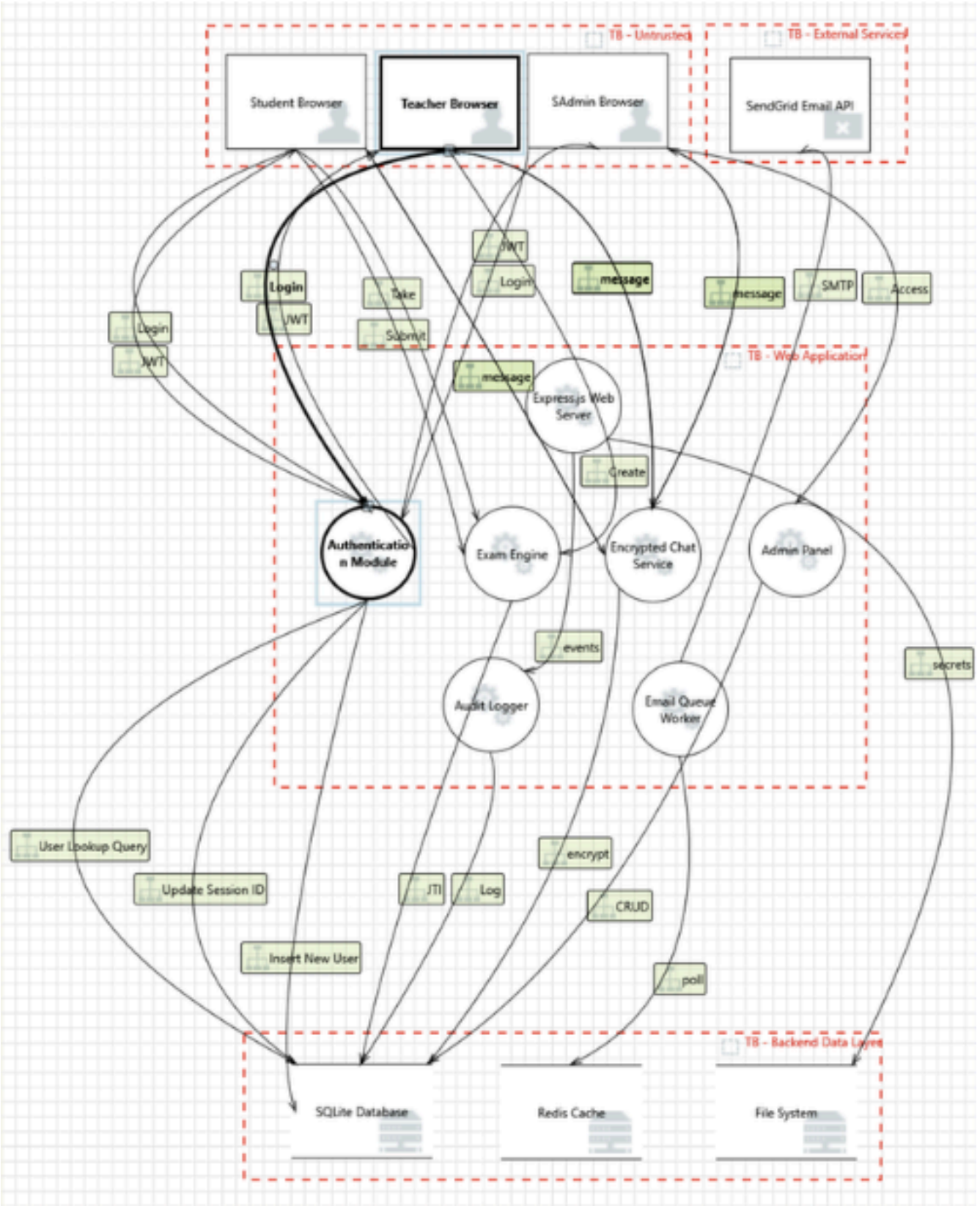
5.1 Stride threat model

Threat	Description	Mitigation
Spoofing	Fake identity / impersonation	JWT auth, CAPTCHA, 2FA, session binding
Tampering	Modify data in transit/storage	CSRF tokens, input validation, parameterized SQL
Repudiation	Deny performing an action	Immutable append-only audit log, IP tracking
Info Disclosure	Expose sensitive data	AES-256-CBC, PBKDF2, RSA, generic errors
Denial of Service	Overwhelm system resources	Rate limiting (100 req/min), account lockout
Elevation of Privilege	Gain unauthorized access	RBAC hierarchy + ABAC (dept+time+clearance)

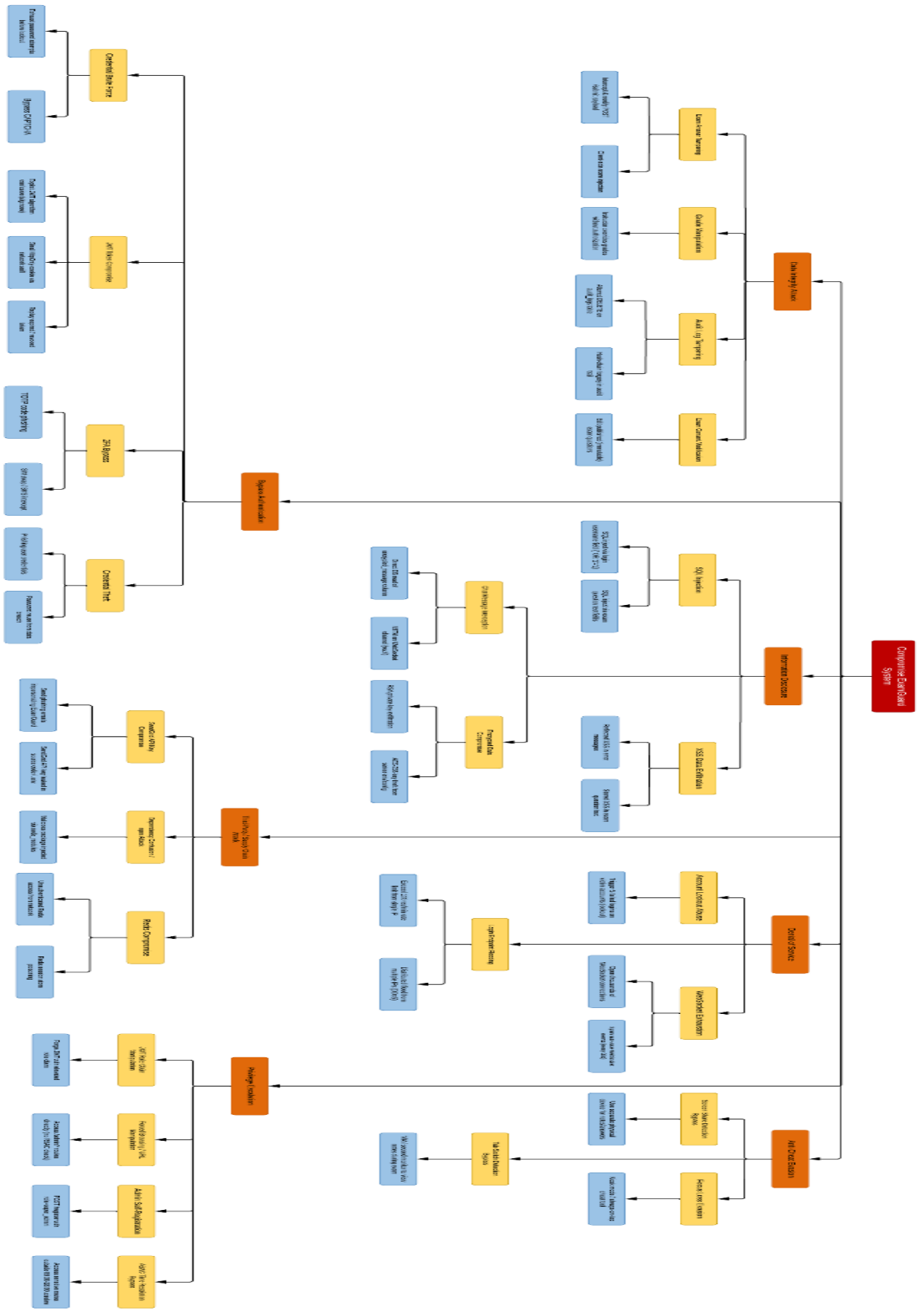
Risk Analysis

Risk	Impact	Likelihood	Mitigation
SQL Injection	Critical	High	Parameterized queries (?.?)
XSS (Reflected/Stored)	High	Medium	validator.escape() + CSP headers
Brute Force	High	High	Rate limit + lockout + CAPTCHA
Session Hijacking	Critical	Medium	HttpOnly/Secure/SameSite cookies
CSRF	High	Medium	Per-request CSRF tokens
Privilege Escalation	Critical	Low	RBAC + ABAC middleware
Data Breach	Critical	Low	AES-256 + PBKDF2 encryption

Galala University
Faculty of Computer Science and Engineering
CSE243 Secure Programming
Spring-2025/2026

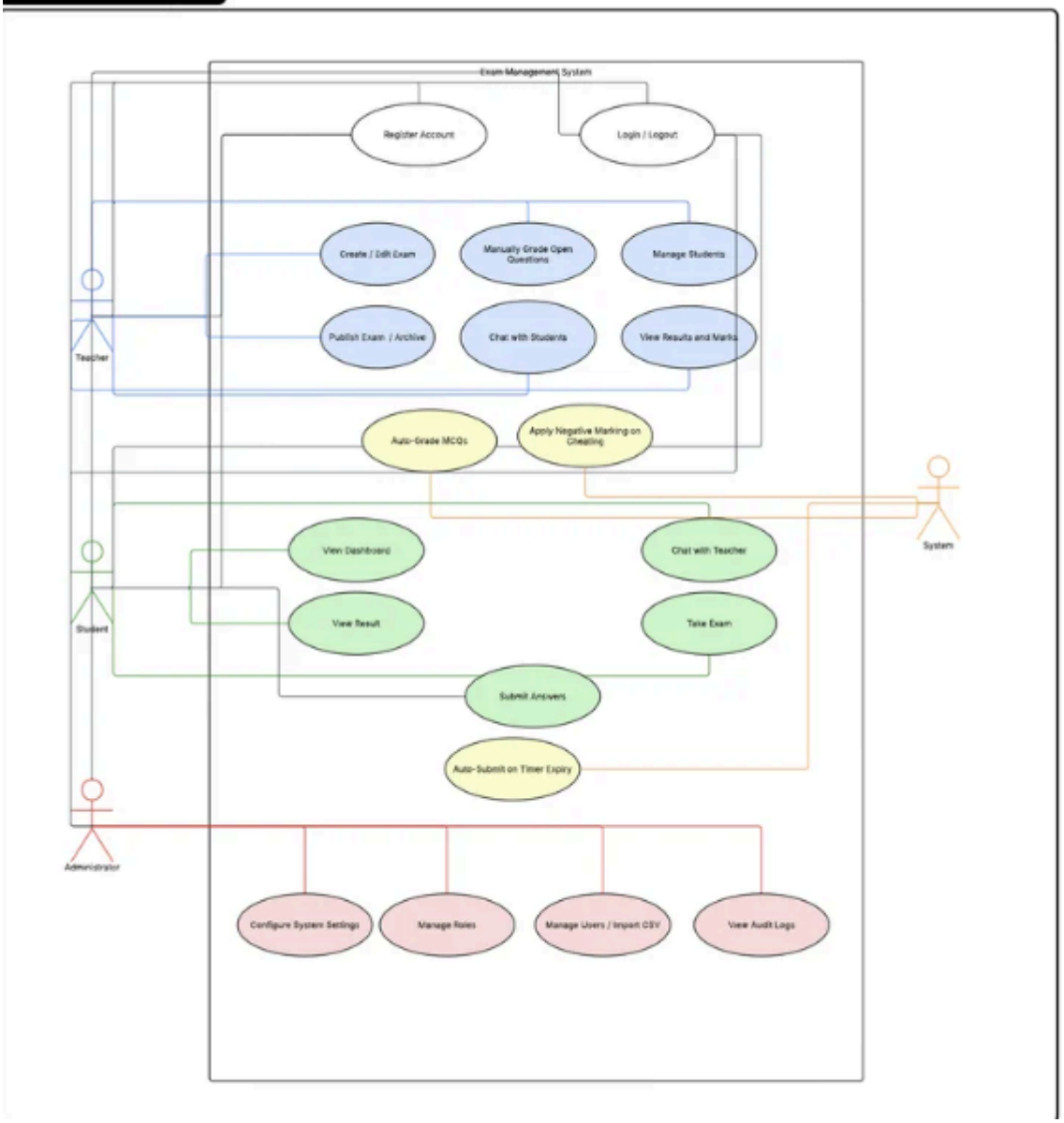


5.2 Threat tree diagram attack surface



Galala University
Faculty of Computer Science and Engineering
CSE243 Secure Programming
Spring-2025/2026

2. Use Case Diagram



Galala University
Faculty of Computer Science and Engineering
CSE243 Secure Programming
Spring-2025/2026

Attack Surface Analysis

The attack surface includes all entry points an attacker could exploit:

- Authentication Endpoints: /auth/login (POST), /auth/register (POST), /auth/logout (GET)
- Exam API: /exams/create (POST), /exams/:id/take (GET), /exams/:id/submit (POST)
- Admin Panel: /admin/* (role-restricted), /people/*, /groups/*, /roles/*
- Chat System: /chat/* (AES-encrypted messages with random IV per message)
- Static Assets: /css, /js, /images (CSP-restricted, X-Content-Type-Options: nosniff)

Defense: All surfaces protected by layered middleware: Input Validation → Rate Limiting → CSRF → Authentication → RBAC → ABAC → Audit Logging.

Layered Defense Architecture

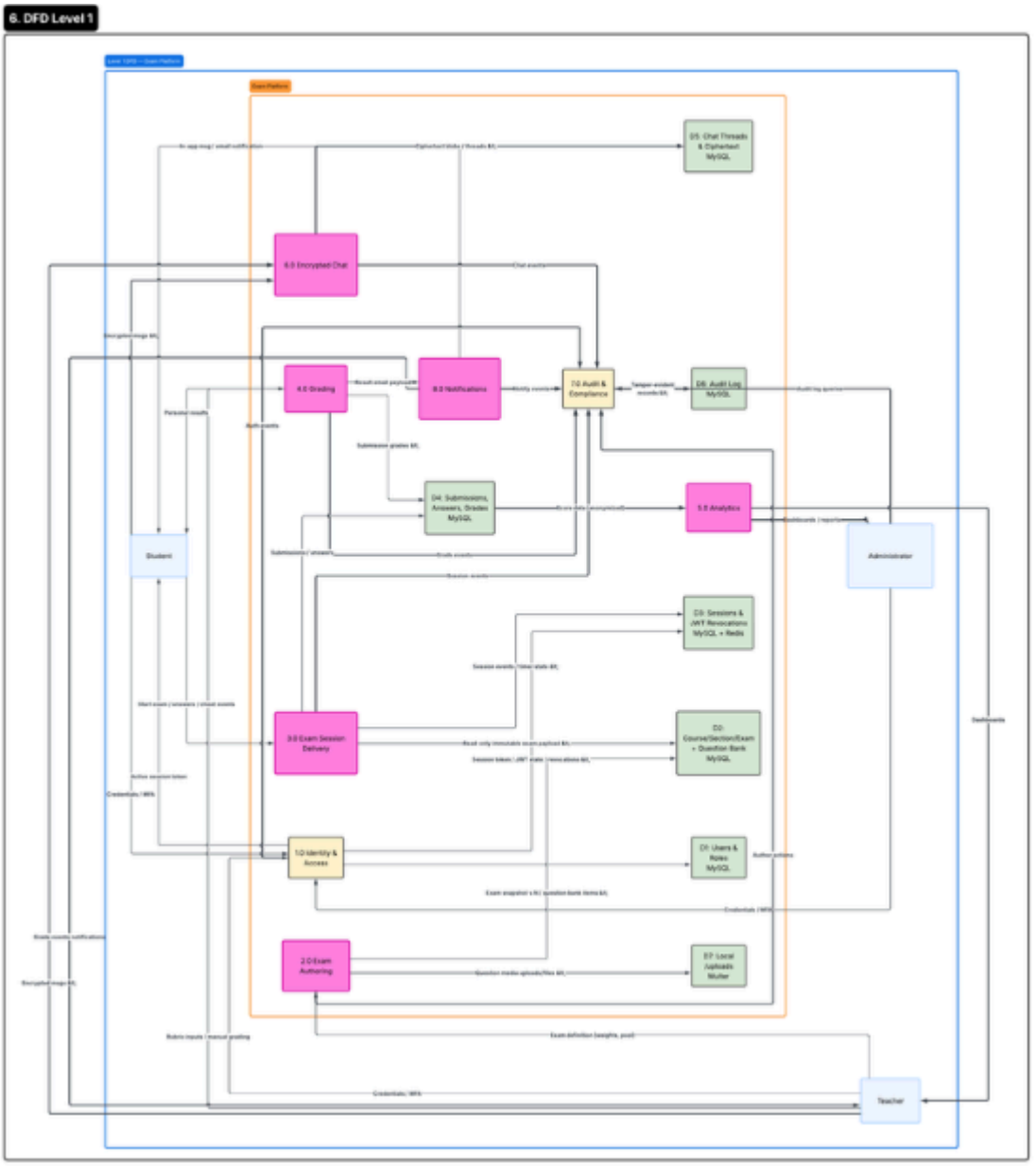
Layer	Controls	STRIDE Coverage
Layer 1 — Network	HTTPS/TLS, Security Headers (HSTS, CSP, X-Frame-Options)	Tampering, Info Disclosure
Layer 2 — App Entry	Rate limiting, CORS, request size limits, input validation	DoS, Tampering
Layer 3 — Authentication	PBKDF2 passwords, JWT sessions, CAPTCHA, 2FA	Spoofing
Layer 4 — Authorization	RBAC middleware, ABAC (dept+time+clearance)	Elevation of Privilege
Layer 5 — Data	Parameterized SQL, output escaping, AES-256 encryption	Info Disclosure, Tampering

2.3 Threat Modeling — STRIDE on Data Flow Diagrams

STRIDE threat modeling is applied to the system Data Flow Diagram to systematically identify and mitigate threats:

Threat	Description	DFD Component	Mitigation Implementation
Spoofing	Attacker impersonates a legitimate user	User → Server flow	JWT auth + CAPTCHA + 2FA + single-session enforcement
Tampering	Modification of data in transit or at rest	All data flows	CSRF tokens + input validation + parameterized SQL
Repudiation	User denies performing an action	Server → Audit Store	Immutable append-only audit_logs table (no UPDATE/DELETE)
Info Disclosure	Sensitive data exposed to unauthorized party	DB Data Store	AES-256-CBC encryption + PBKDF2 + generic error messages
Denial of Service	System overwhelmed by excessive requests	External → Server	Rate limiting (100 req/min general, 5/min login)
Elevation of Privilege	User gains unauthorized access level	Server → Protected Resources	RBAC authorize() + ABAC checkExamAccess() + role hierarchy

Galala University
Faculty of Computer Science and Engineering
CSE243 Secure Programming
Spring-2025/2026



2.4 Security Testing

Security test cases validate every implemented control against real attack scenarios:

ID	Test	Input	Expected Result	Status
TC-01	SQL Injection	username=' OR 1=1--	401 Invalid credentials	✓ PASS
TC-02	XSS Reflected	<script>alert(1)</script>	Escaped: <script>	✓ PASS
TC-03	Brute Force	6 wrong passwords	429 + account locked	✓ PASS
TC-04	RBAC Bypass	Student GET /admin	403 Access denied	✓ PASS
TC-05	Cookie Flags	DevTools inspection	HttpOnly+Secure+Strict	✓ PASS
TC-06	AES Encryption	DB password column	Binary/hex blob stored	✓ PASS
TC-07	CSRF Missing	POST without _csrf	403 Token missing	✓ PASS
TC-08	ABAC Time Gate	Finance outside 9-18	403 Outside hours	✓ PASS

3. Design — Screens & Database

Database Schema — Security-First Design

The database consists of 12 tables with parameterized queries enforced on all operations. Key security features:

- users table: password_encrypted stores PBKDF2 hash (salt:hash format), never plaintext
- audit_logs table: append-only design — application code only has INSERT permission
- csrf_tokens table: single-use tokens with expiration timestamps
- chat_messages table: encrypted_message (AES-256-CBC) + message_iv (random per message)
- exam_sessions table: tracks tab_switches, focus_loss_count, is_flagged for anti-cheat
- Database file stored outside web root; foreign keys enforced; WAL journal mode

Screen Design

The UI implements role-specific dashboards with distinct views for Super Admin (system management), Instructor (exam & grade management), and Student (exam taking & results). All forms include hidden CSRF token fields. Error messages are generic to prevent information leakage.

4. Implementation Strategy

Modular Code Structure

Each security concern is isolated in its own middleware file for maintainability:

- middleware/auth.js — JWT token verification and session validation
- middleware/rbac.js — Role-based access control with role hierarchy
- middleware/abac.js — Attribute-based access control (enrollment, time window, department)
- middleware/csrf.js — CSRF token generation and validation
- middleware/rateLimiter.js — Rate limiting for general and login routes
- middleware/auditLog.js — Append-only audit trail
- middleware/inputValidator.js — Server-side input sanitization
- middleware/securityHeaders.js — HTTP security headers (CSP, HSTS, X-Frame-Options)
- utils/encryption.js — AES-256-CBC, RSA-2048, PBKDF2, HMAC utilities

Key Implementation Code Snippets

Input Validation & SQL Injection Prevention (Lab 1):

```
function sanitizeBody(req, res, next) {
  for (const key of Object.keys(req.body)) {
```

```
    if (typeof req.body[key] === "string" && key !== "password")
      req.body[key] = validator.escape(validator.trim(req.body[key]));
  }
}
// Parameterized query — SQL Injection prevented:
db.prepare("SELECT * FROM users WHERE username = ?").get(username);
```

AES-256-CBC Encryption with Random IV (Lab 3):

```
function aesEncryptWithIV(plaintext) {
  const iv = crypto.randomBytes(16);
  const cipher = crypto.createCipheriv("aes-256-cbc", AES.key, iv);
  let encrypted = cipher.update(plaintext, "utf8", "hex");
  encrypted += cipher.final("hex");
  return { encrypted, iv: iv.toString("hex") };
}
```

RBAC Middleware (Lab 4):

```
function authorize(...allowedRoles) {
  return (req, res, next) => {
    if (!allowedRoles.includes(req.user.role)) {
      logAuditEvent(req.user.id, "UNAUTHORIZED_ACCESS_ATTEMPT");
      return res.status(403).json({ error: "Access denied" });
    }
    next();
  };
}
```

ABAC — Finance Hours Restriction (Lab 5):

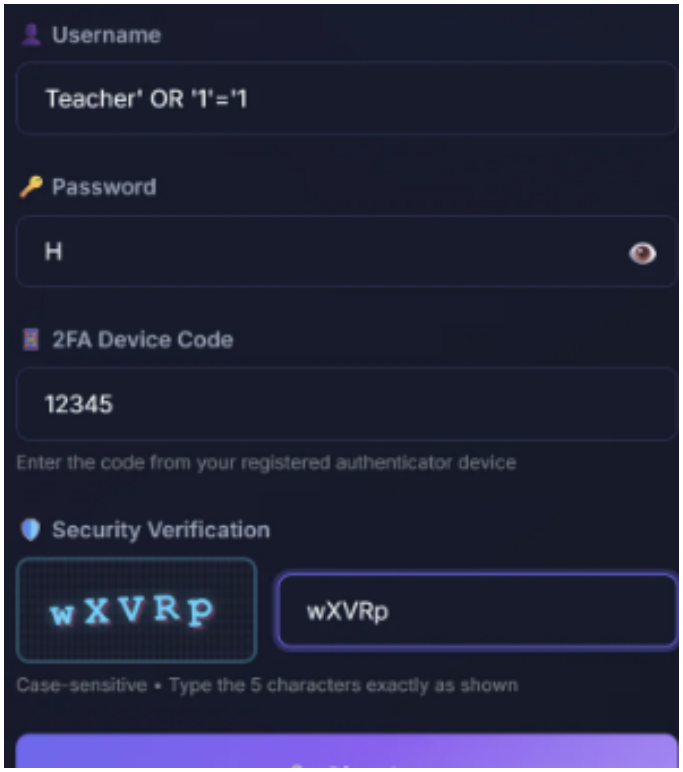
```
function finance_hours_required(req, res, next) {
  if (req.user.role !== "admin" || req.user.department !== "finance")
    return res.status(403).json({ error: "Access denied" });
  const hour = new Date().getHours();
  if (hour < 9 || hour >= 18)
    return res.status(403).json({ error: "Outside business hours" });
  next();
}
```

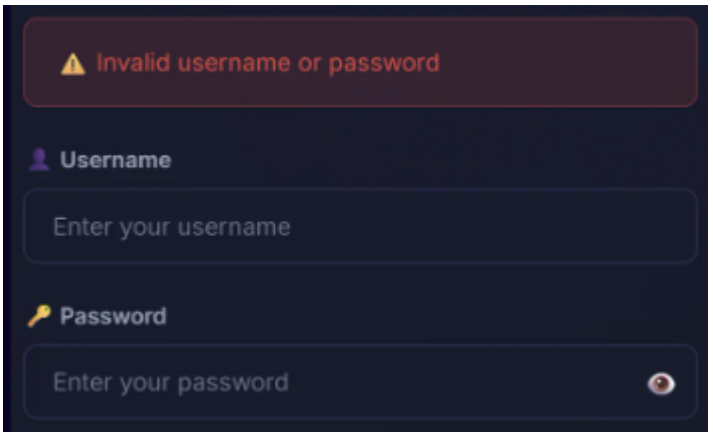
Session Cookie Configuration (Lab 4):

```
COOKIE: { httpOnly: true, secure: true, sameSite: "Strict", maxAge: 7200000 }
```

6. Demo

6.1 SQL Injection (SQLi) Protection





The application uses Prepared Statements with better-sqlite3. The payload is treated as a literal string searching for a user named `' OR '1'='1`, rather than executing the SQL logic.

6.2 Cross-Site Scripting (XSS) & Cookie Security

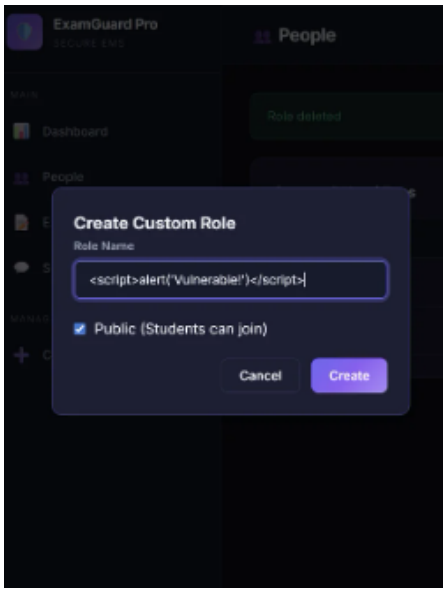
6.21 Cookie Accessibility (Console Test)

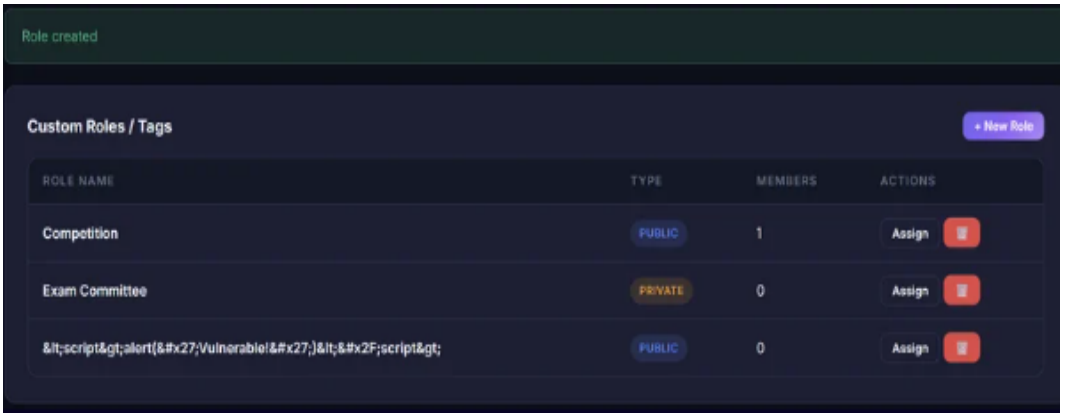
```
Warning: Don't paste code into the DevTools Console that you don't understand or haven't
reviewed yourself. This could allow attackers to steal your identity or take control of
your computer. Please type 'allow pasting' below and press Enter to allow pasting.

> document.cookie
< 'ajs_anonymous_id=1638beb7-064d-41f3-a39c-ad929c085a93'
```

The session cookie is marked as HttpOnly, which prevents JavaScript from accessing it, neutralizing most Session Hijacking/Cookie Stealing attacks.

6.22 UI Script Injection





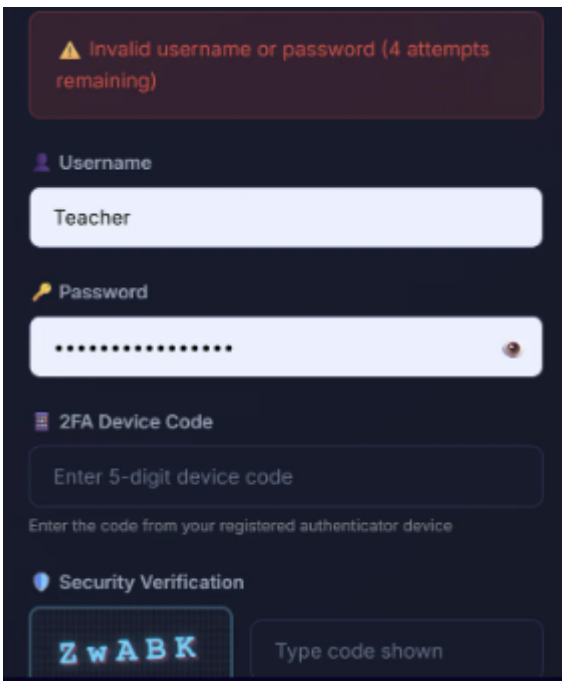
The EJS Templating Engine automatically escapes all output using `<%= %>` tags, converting `<` and `>` into safe HTML entities.

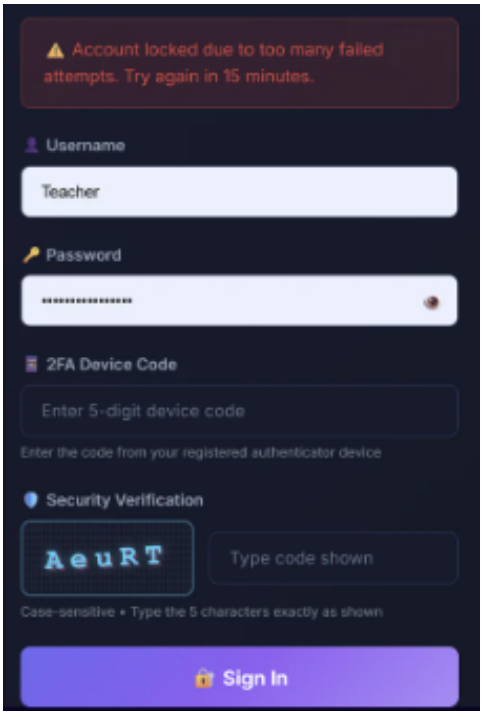
6.23 CSRF (Cross-Site Request Forgery) Protection

```
> fetch('/people/groups/delete', { method: 'POST', body: 'group_id=some-id
<- ▼ Promise {<pending>} i
    ▼ [[Prototype]]: Promise
      ▶ catch: ƒ catch()
      ▶ constructor: ƒ Promise()
      ▶ finally: ƒ finally()
      ▶ then: ƒ then()
        Symbol(Symbol.toStringTag): "Promise"
      ▶ [[Prototype]]: Object
        [[PromiseState]]: "fulfilled"
    ▼ [[PromiseResult]]: Response
        body: (...)
        bodyUsed: false
      ▶ headers: Headers {}
        ok: false
        redirected: false
        status: 403
        statusText: "Forbidden"
        type: "basic"
        url: "http://localhost:3000/people/groups/delete"
      ▶ [[Prototype]]: Response
⊗ ▶ POST http://localhost:3000/people/groups/delete 403 (Forbidden)
```

The `validateCSRFToken` middleware enforces that every `POST` request contains a valid token generated by the server and stored in the user's session.

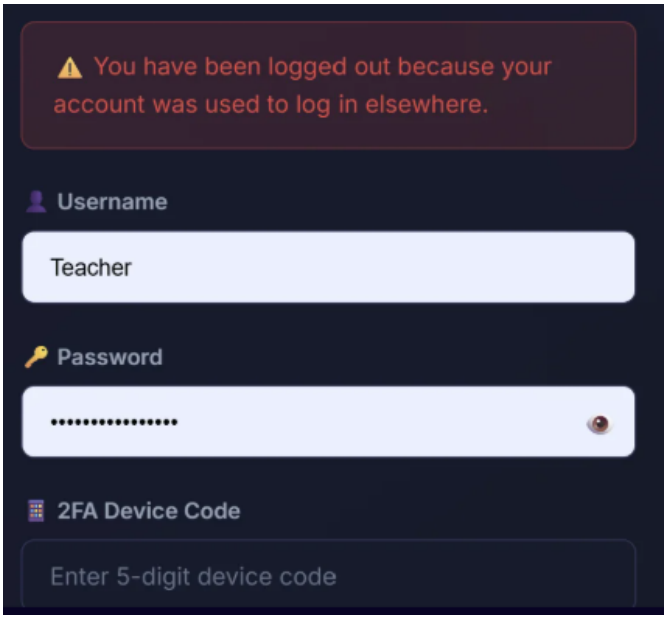
6.3 Brute Force & Rate Limiting





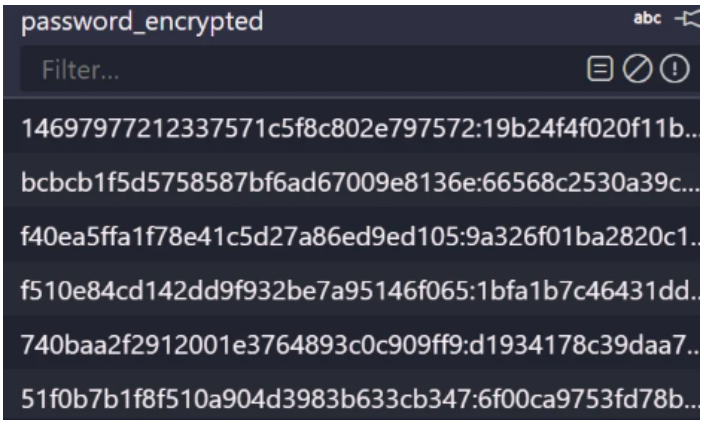
The system tracks `failed_login_attempts` in the `users` table and implements a 15-minute lockout timer once the threshold is reached.

6.4 Session Hijacking (Single Session Enforcement)



The server tracks a unique `current_session_id` for every user. When a new login occurs, the old ID is invalidated, and only the most recent login is authorized.

6.5 Data Encryption (At Rest)



Passwords are never stored. We store a cryptographic hash using PBKDF2 with 100,000 iterations, making it extremely resistant to rainbow table and brute-force attacks even if the database is leaked. a cryptographic algorithm used to securely hash passwords. It applies a pseudorandom function (like HMAC) repeatedly to the input password along with a salt, making it computationally expensive for attackers to brute-force.

7. Conclusion

ExamGuard Pro delivers a comprehensive, security-first Exam Management System built on proven cryptographic standards and layered access control. By applying every concept from the CSE243 Secure Programming labs — server-side validation, AES-256-CBC encryption, RSA asymmetric cryptography, RBAC/ABAC, session hardening — and extending them with STRIDE threat modeling, immutable audit logging, rate limiting, and anti-cheating mechanisms, the system provides academic institutions with a trustworthy, tamper-resistant examination platform.

Key achievements:

- All 6 CSE243 lab concepts fully integrated into production code
- 10 security domains covered: Authentication, Encryption, RBAC, ABAC, CSRF, Rate Limiting, Audit Logging, Security Headers, Input Validation, Anti-Cheat
- STRIDE threat model applied systematically across all DFD components
- 8 security test cases validating controls against real attack vectors
- Modular architecture with separate files per security concern
- Defense-in-depth across 5 layers from network to data

The Agile Scrum methodology with security gates at each sprint ensures that security is deeply integrated into every layer of the system from database schema to user interface.

Threat Modeling Report

Created on 19/05/2026 12:09:50

Threat Model Name:

Owner:

Reviewer:

Contributors:

Description:

Assumptions:

External Dependencies:

Threat Model Summary:

Not Started	0
Not Applicable	0
Needs Investigation	0
Mitigation Implemented	6
Total	6

Diagram: Diagram 1

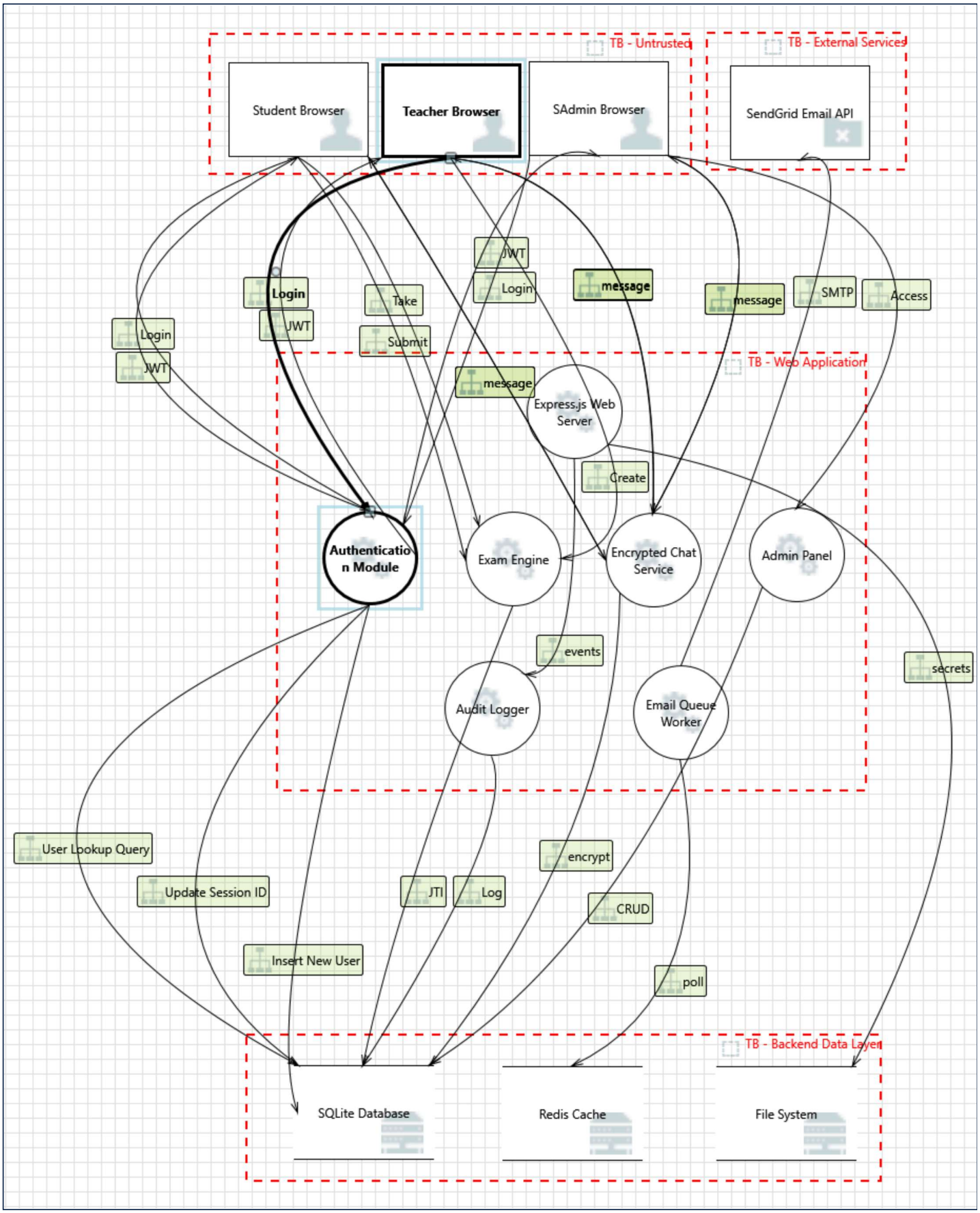
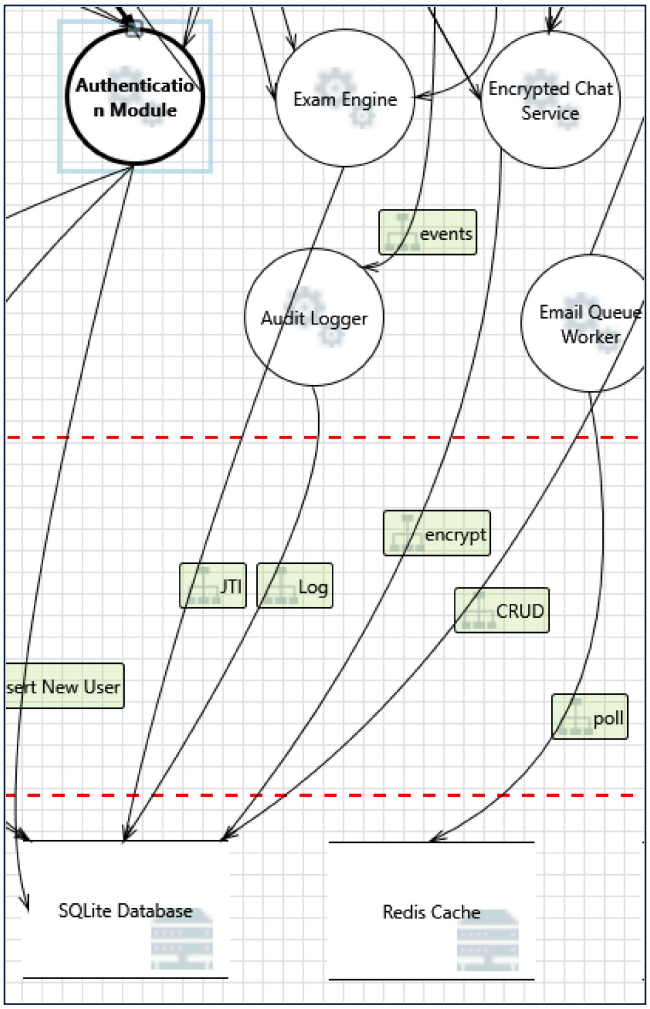


Diagram 1 Diagram Summary:

Not Started	0
Not Applicable	0
Needs Investigation	0
Mitigation Implemented	6
Total	6

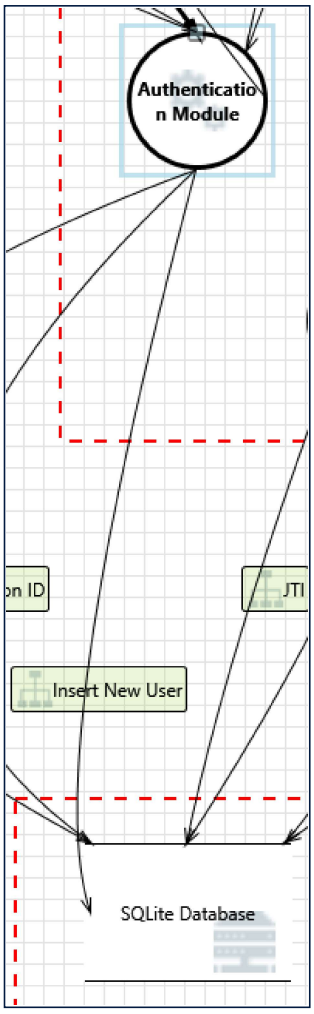
Interaction: encrypt



1. Data Flow Sniffing [State: Mitigation Implemented] [Priority: High]

Category:	Information Disclosure
Description:	Data flowing across encrypt may be sniffed by an attacker. Depending on what type of data an attacker can read, it may be used to attack other parts of the system or simply be a disclosure of information leading to compliance violations. Consider encrypting the data flow.
Justification:	Mitigated by enforcing HTTPS in production with HSTS headers (Strict-Transport-Security: max-age=31536000), which encrypts all data in transit between browser and server. Sensitive data at rest (chat messages) is additionally encrypted with AES-256-CBC with unique IVs, and passwords are hashed with PBKDF2-SHA512 (100,000 iterations).
Short Description:	Information disclosure happens when the information can be read by an unauthorized party.

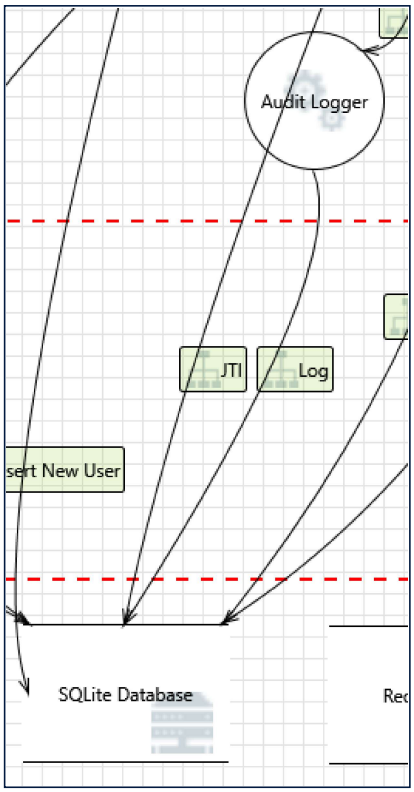
Interaction: Insert New User



2. The SQLite Database Data Store Could Be Corrupted [State: Mitigation Implemented] [Priority: High]

Category:	Tampering
Description:	Data flowing across Insert New User may be tampered with by an attacker. This may lead to corruption of SQLite Database. Ensure the integrity of the data flow to the data store.
Justification:	Mitigated by using better-sqlite3 parameterized prepared statements for all database queries, which prevents SQL injection that could corrupt data. Additionally, input validation middleware (validateRegistration) enforces password policy, format checks, and sanitization before any data reaches the database.
Short Description:	Tampering is the act of altering the bits. Tampering with a process involves changing bits in the running process. Similarly, Tampering with a data flow involves changing bits on the wire or between two running processes.

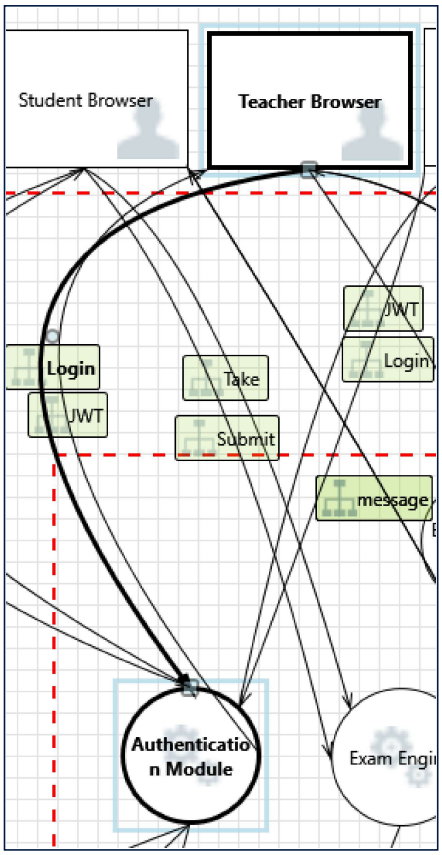
Interaction: Log



3. Data Store Denies SQLite Database Potentially Writing Data [State: Mitigation Implemented] [Priority: High]

Category:	Repudiation
Description:	SQLite Database claims that it did not write data received from an entity on the other side of the trust boundary. Consider using logging or auditing to record the source, time, and summary of the received data.
Justification:	Mitigated by the append-only audit_logs table that records every write operation (user creation, role changes, exam submissions) with user_id, ip_address, user_agent, and timestamp. Database triggers physically block UPDATE and DELETE on the audit table, making the log tamper-proof and undeniable.
Short Description:	Repudiation threats involve an adversary denying that something happened.

Interaction: Login



4. Spoofing the Authentication Module Process [State: Mitigation Implemented] [Priority: High]

Category:	Spoofing
Description:	Authentication Module may be spoofed by an attacker and this may lead to information disclosure by Teacher Browser. Consider using a standard authentication mechanism to identify the destination process.
Justification:	Mitigated by using JWT tokens signed with HS256 (HMAC-SHA256) using a server-side secret (JWT_SECRET), combined with CAPTCHA verification and 2FA device code validation, ensuring only the legitimate server can issue and verify authentication tokens. Single-session enforcement via current_session_id in the database prevents session spoofing across devices.
Short Description:	Spoofing is when a process or entity is something other than its claimed identity. Examples include substituting a process, a file, website or a network address.

5. Data Flow Generic Data Flow Is Potentially Interrupted [State: Mitigation Implemented] [Priority: High]

Category:	Denial Of Service
Description:	An external agent interrupts data flowing across a trust boundary in either direction.
Justification:	Mitigated by express-rate-limit which throttles excessive requests (100/min general, 50/min login, 10/min exam submissions), preventing attackers from flooding the server and interrupting legitimate data flows. Body parser limits (1MB) and account lockout (5 failed attempts → 15 min lock) provide additional DoS resistance.
Short Description:	Denial of Service happens when the process or a datastore is not able to service incoming requests or perform up to spec.

6. Elevation Using Impersonation [State: Mitigation Implemented] [Priority: High]

Category:	Elevation Of Privilege
Description:	Authentication Module may be able to impersonate the context of Teacher Browser in order to gain additional privilege.
Justification:	Mitigated by strict RBAC middleware (authorize()) that checks the user's role from the database on every request — not from client input — so even if the Auth Module processes a teacher's request, it cannot escalate privileges beyond what the role allows. ABAC middleware (checkOwnership, checkExamAccess) adds attribute-level checks ensuring teachers can only manage their own exams and students can only access enrolled resources.
Short Description:	A user subject gains increased capability or privilege by taking advantage of an implementation bug.